

Easy Performant Outline 2D | 3D

(高效轻量描边效果, 支持 URP / HDRP)

Easy Performant Outline 是一款工具, 用于在 Unity 中为物体添加可自定义的 2D 和 3D 描边效果。该工具支持多种渲染器, 并经过优化, 可在不影响性能的情况下生成高质量的描边。无论是 2D 还是 3D 项目, 都可以通过颜色、厚度、样式等可配置选项, 提高游戏和应用中物体的视觉清晰度。

目录

1. [简介](#)
2. [安装](#)
 - [将资源导入项目](#)
 - [针对不同渲染器的设置](#)
 - [内置渲染器设置](#)
 - [URP 渲染器设置](#)
 - [HDRP 渲染器设置](#)
 - [安装问题排查](#)
3. [核心组件](#)
 - [Outliner 设置](#)
 - [Outlinable 设置](#)
4. [高级设置](#)
 - [遮罩](#)
 - [2D 功能](#)
5. [集成](#)
 - [使用 C# API](#)
 - [DOTween 支持](#)
6. [常见问题\(FAQ\)](#)
7. [联系信息](#)

如果你对 EP Outline 有任何疑问, 请先查看常见问题 (FAQs), 你可能可以在那里找到答案。
如果仍然无法解决问题, 欢迎提交问题反馈, 或通过电子邮件 pirate.parrot.software@gmail.com, 或 **WeChat ID: thepirateparrot** 与我们联系。

在报告 Bug 时, 如果能提供清晰的复现步骤, 将对我们有很大帮助, 这样可以更快地定位并修复问题。

如果回复需要一些时间, 请多多包涵—我们每天会收到大量消息, 但会尽快回复你。
最后, 如果你有功能想法或改进建议, 欢迎随时与我们分享。
我们的一些最佳功能正是直接来自用户的反馈!

★ 我们非常期待你的反馈! ★

如果你喜欢使用 Easy Performant Outline, 欢迎在 [Unity Asset Store](#) 上留下评价!

你的评分和反馈将帮助我们改进该资源, 并持续开发新功能。

感谢你的支持! 🙏

简介

Easy Performant Outline 是一款用于在 Unity 中为 2D 和 3D 对象创建高质量描边效果的工具。该资源可以让你轻松为对象添加描边，用于突出场景中的物体、改善视觉感知并增强交互性。它通过以最低的资源消耗实现描边效果来解决性能问题，使其成为对性能要求较高项目的理想选择。

主要特性：

- 易于使用：只需几分钟即可完成描边设置。
 - 支持多种 Unity 渲染器 (Built-in、URP、HDRP)。
 - 即使在使用大量描边的情况下，也能保持高性能。
 - 描边参数完全可自定义：颜色、厚度、模糊等。
 - 支持与 DoTween 配合使用，实现动态动画效果。
 - 兼容 2D 和 3D 对象，包括对精灵 (Sprite) 的支持。
 - 支持 Render Graph，适用于现代项目。
-

系统要求：

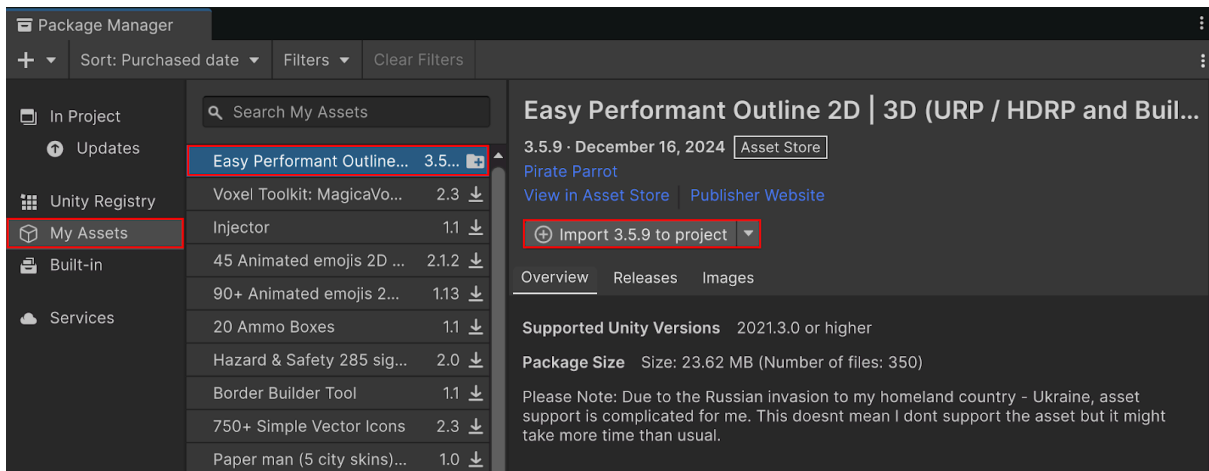
- 支持的平台：Android、iOS、PC、Mac、PS4、Oculus/Vive、WebGL 等。
 - 支持的渲染器：Standard、Universal Render Pipeline (URP)、High Definition Render Pipeline (HDRP)。
 - 硬件要求：该资源针对现代 GPU 进行了优化，但在较复杂的场景中，建议使用至少中端级别的 GPU 以获得最佳性能。
-

安装

将资源导入项目

要通过 Package Manager 安装 Easy Performant Outline, 请按照以下步骤操作：

1. 打开你的 Unity 项目。
2. 前往 **Window > Package Manager**。
3. 切换到 **My Assets**, 找到 **Easy Performant Outline**, 并点击 **Import**。
4. 按照屏幕上的提示完成导入。



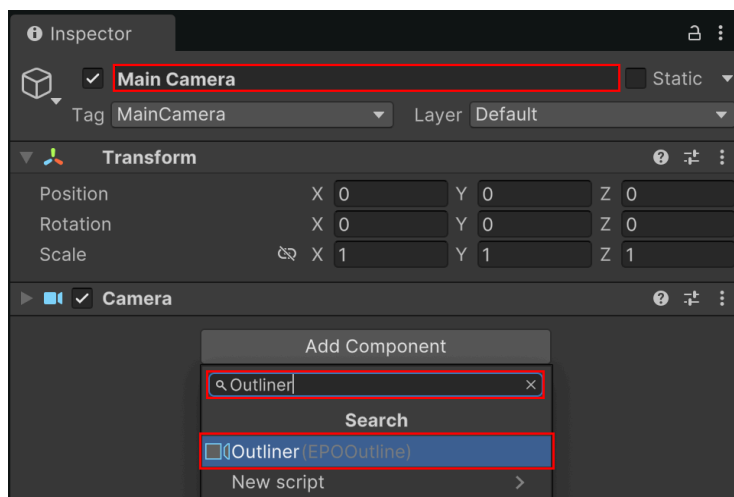
针对不同渲染器的设置

内置渲染器设置

要为内置渲染器 (Built-in Renderer) 配置 Easy Performant Outline, 请按照以下步骤操作:

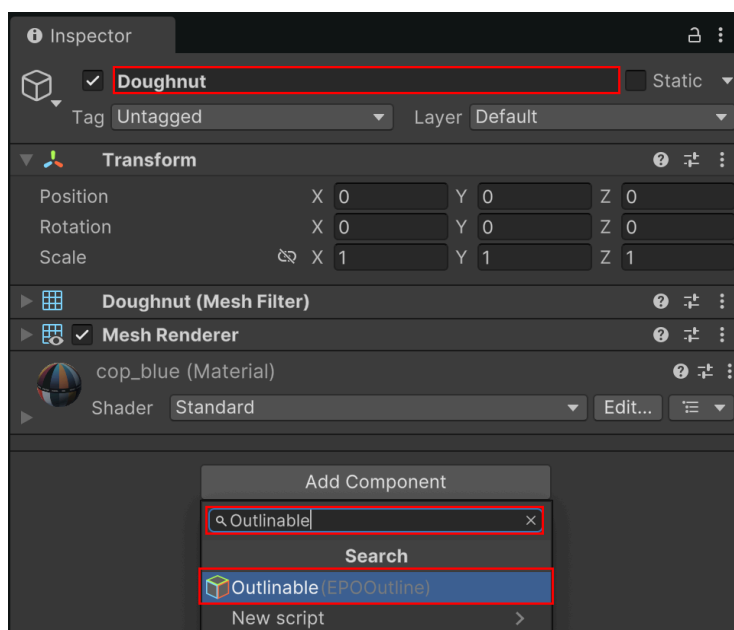
1. 添加 **Outliner** 组件

将 **Outliner** 组件添加到需要渲染描边效果的场景相机上。



2. 添加 **Outlinable** 组件

将 **Outlinable** 组件添加到你希望通过描边效果进行高亮的对象上。



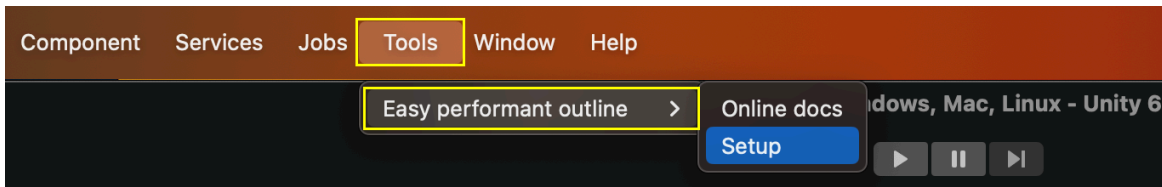
通过这些简单的步骤, 即可在使用内置渲染器的场景中启用描边渲染效果。

URP 渲染器设置

要为通用渲染管线(Universal Render Pipeline, URP)配置 Easy Performant Outline, 请按照以下步骤操作:

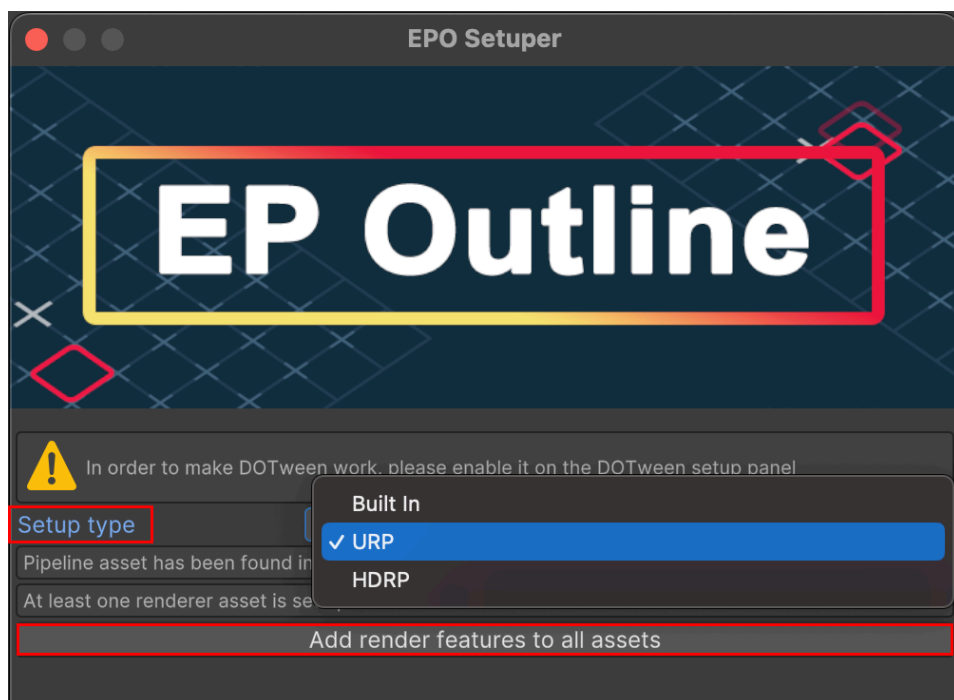
1. 打开设置工具

在 Unity 菜单栏中前往 **Tools > Easy Performant Outline > Setup** , 以打开 EPO 设置工具。



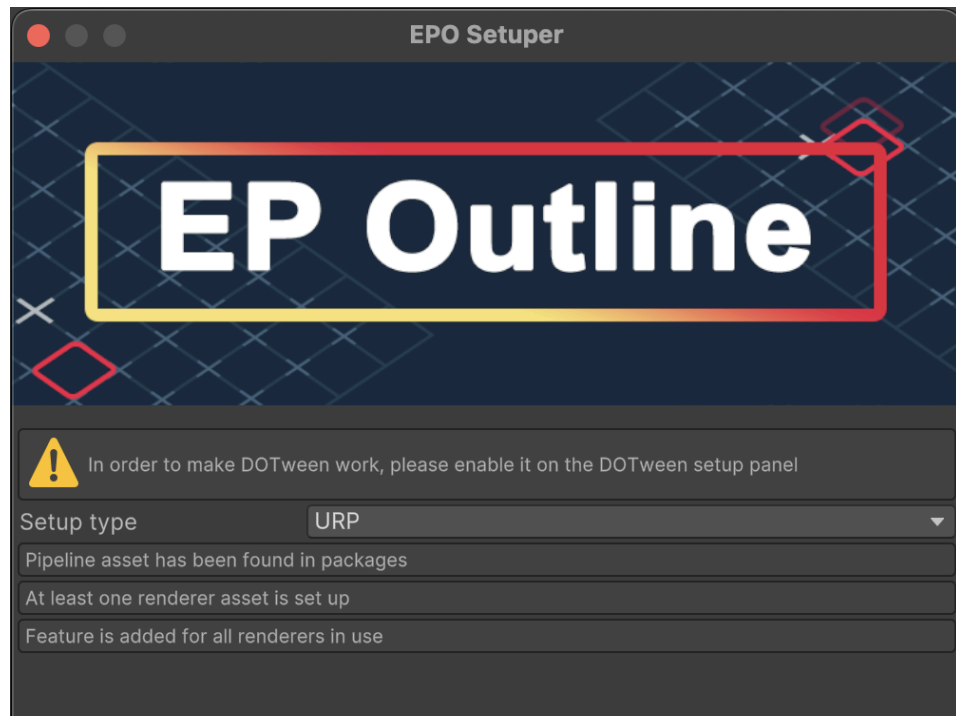
2. 选择 **URP** 并应用 **URP** 描边功能

在设置窗口中, 将渲染管线类型选择为 **URP**, 然后单击 **Add render features to all assets** 按钮。这将自动为你的项目配置 URP 描边功能。



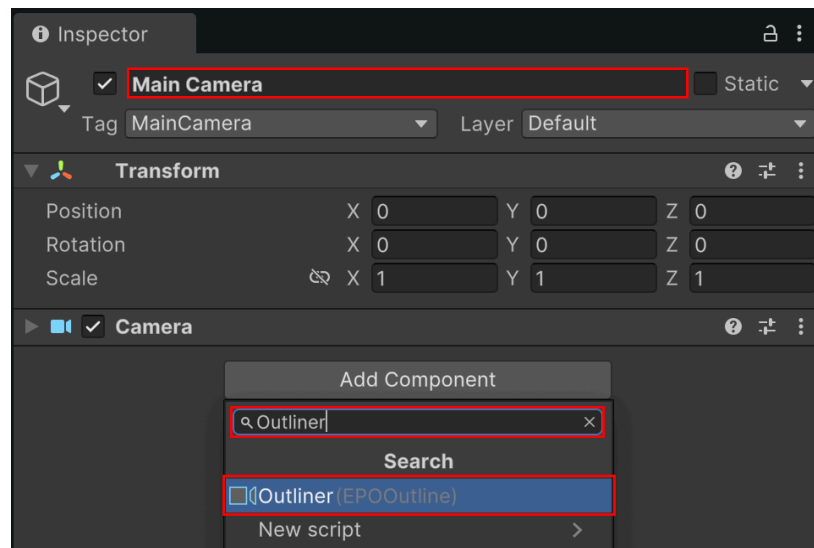
3. 验证设置

确认 URP 描边功能已成功添加。如果检测到渲染管线资源并且配置正确，设置工具将会进行提示。



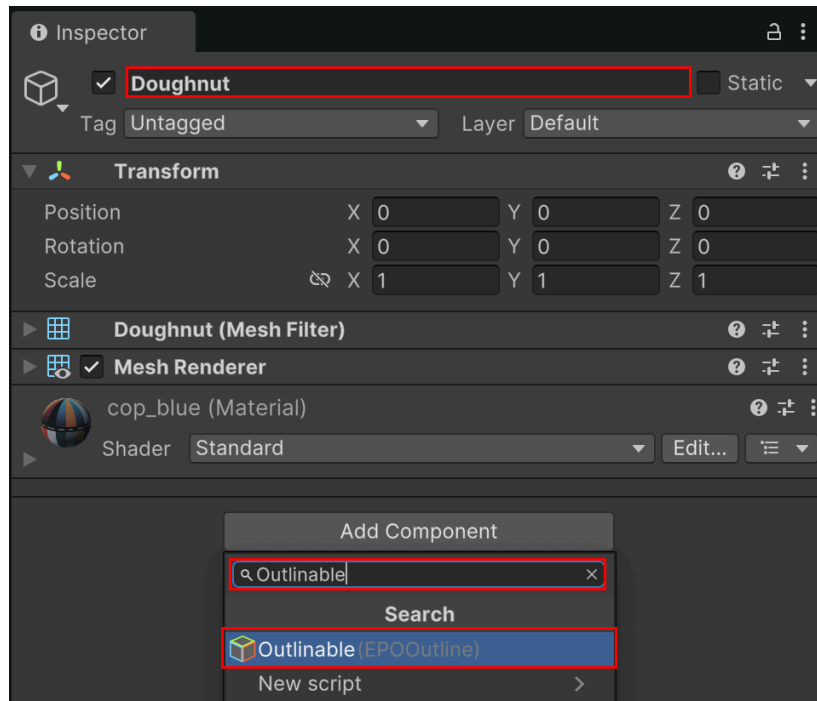
4. 添加 Outliner 组件

将 **Outliner** 组件添加到需要渲染描边效果的场景相机上。



5. 添加 **Outlinable** 组件

将 **Outlinable** 组件添加到你希望通过描边效果进行高亮的对象上。



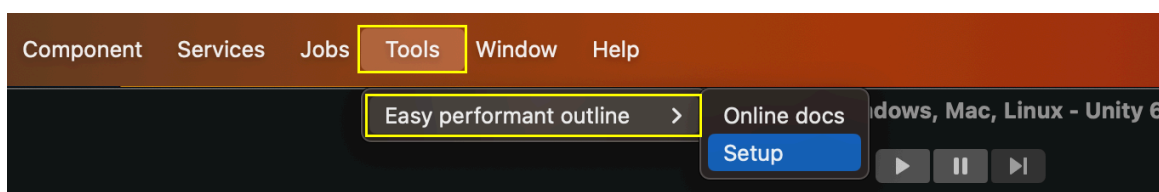
通过这些简单的步骤，即可在使用 URP 渲染器的场景中启用描边渲染效果。

HDRP 渲染器设置

要为高清渲染管线(High Definition Render Pipeline, HDRP)配置 Easy Performant Outline, 请按照以下步骤操作:

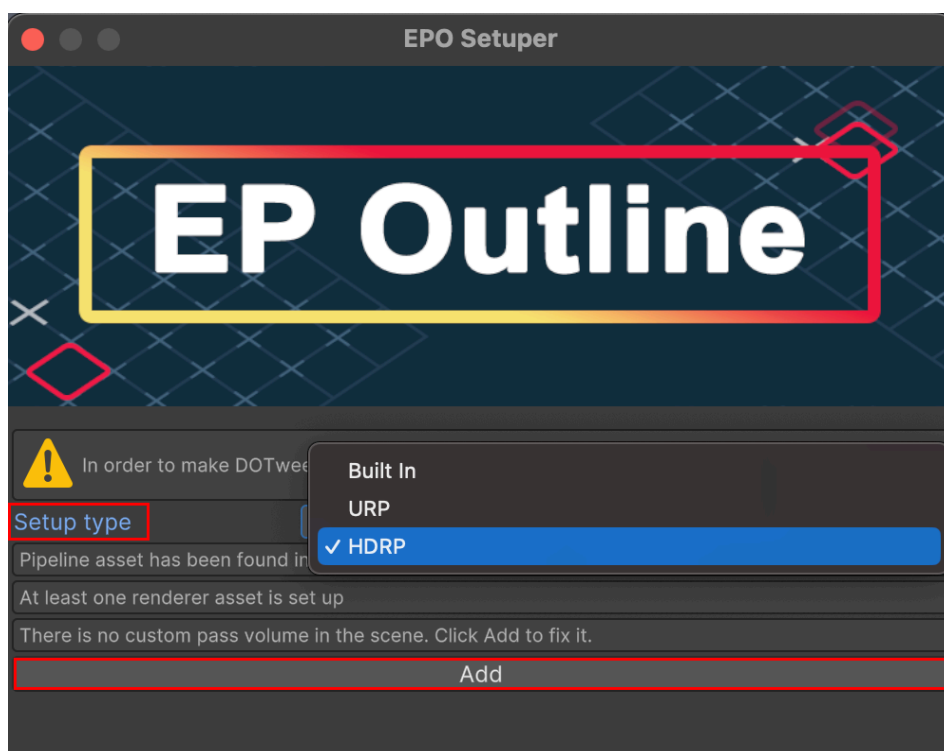
1. 打开设置工具

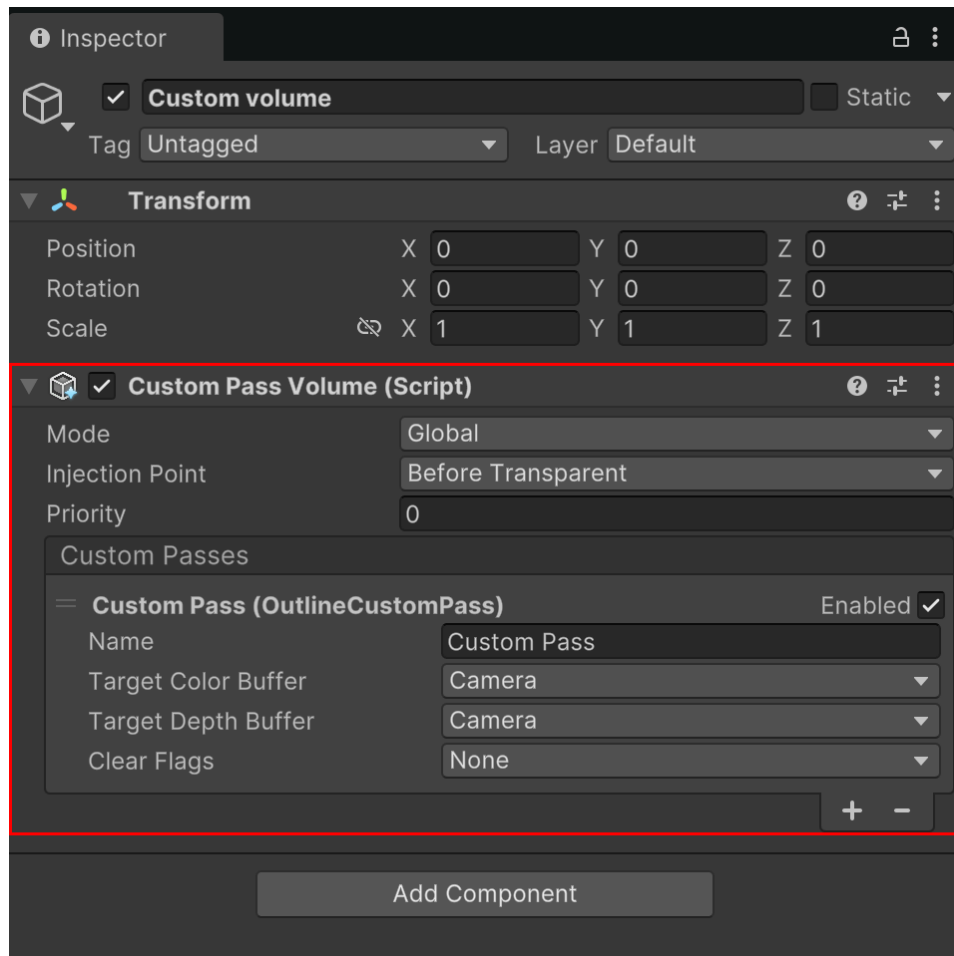
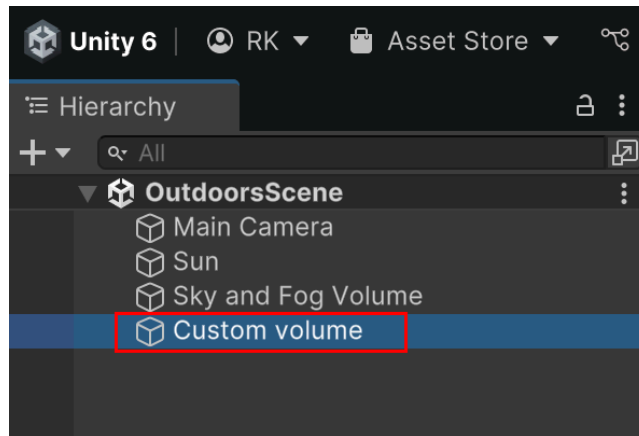
在 Unity 菜单栏中前往 **Tools > Easy Performant Outline > Setup**, 以打开 EPO 设置工具。



2. 选择 HDRP 并创建自定义 Pass

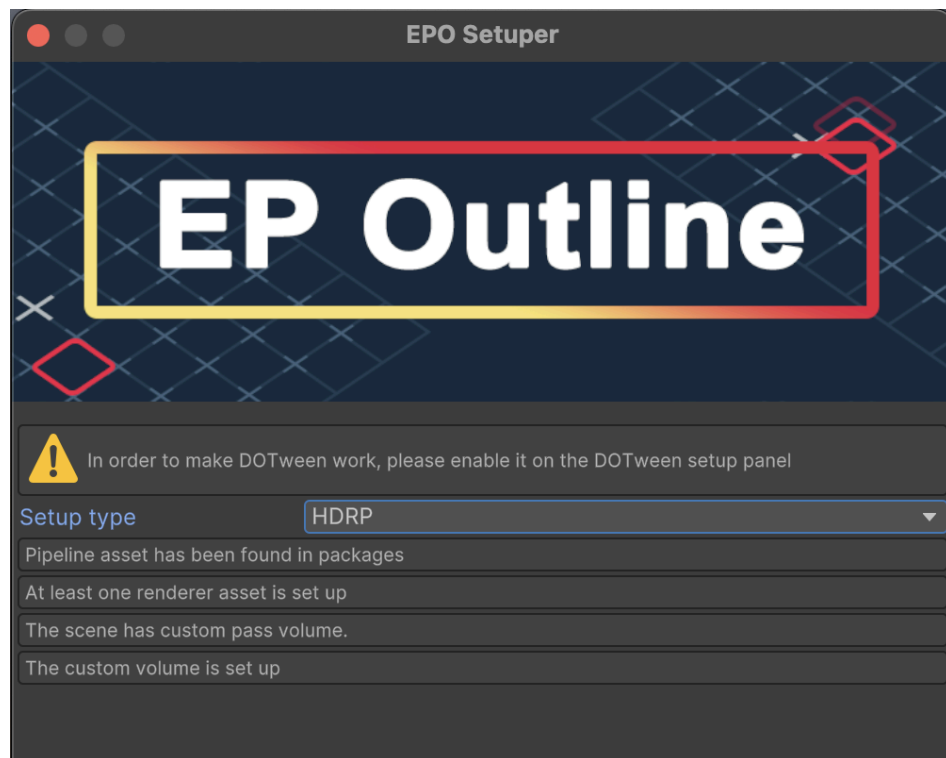
在设置窗口中, 将渲染管线类型选择为 **HDRP**。点击 **Add** 按钮, 在场景中创建一个 **Custom Pass Volume**。随后系统会提示你向新创建的 Volume 添加一个自定义 Pass——再次点击 **Add** 按钮即可完成设置。





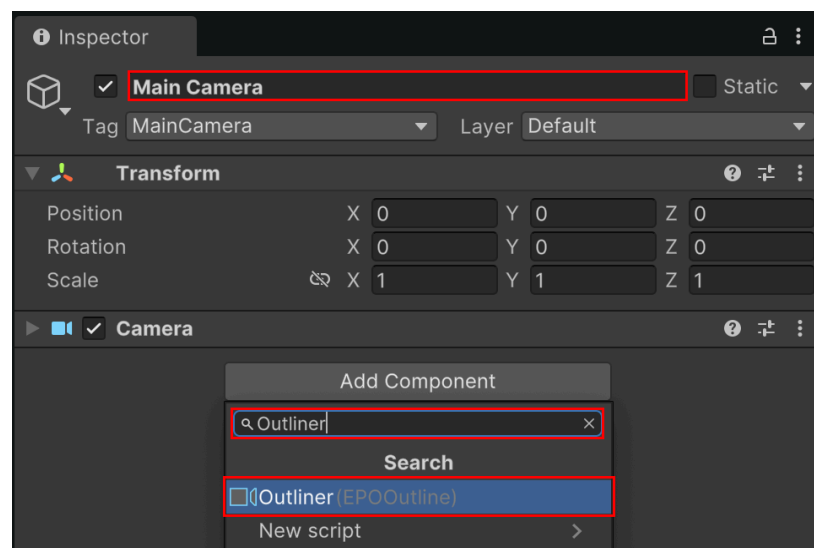
3. 验证设置

确认 HDRP 描边功能已成功添加。如果检测到渲染管线资源并且配置正确，设置工具将会进行提示。



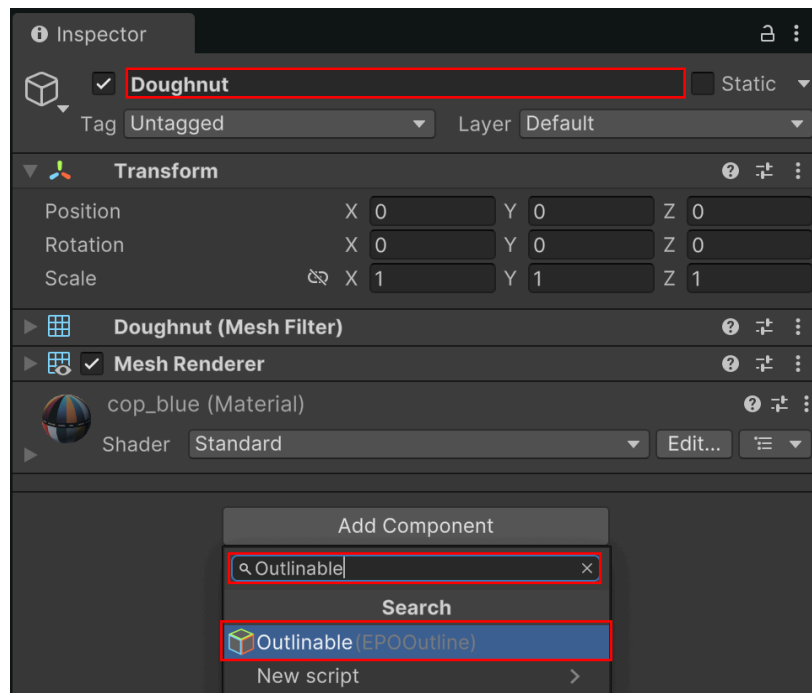
4. 添加 Outliner 组件

将 **Outliner** 组件添加到需要渲染描边效果的场景相机上。



5. 添加 **Outlinable** 组件

将 **Outlinable** 组件添加到你希望通过描边效果进行高亮的对象上。



通过这些简单的步骤，即可在使用 HDRP 渲染器的场景中启用描边渲染效果。

安装问题排查

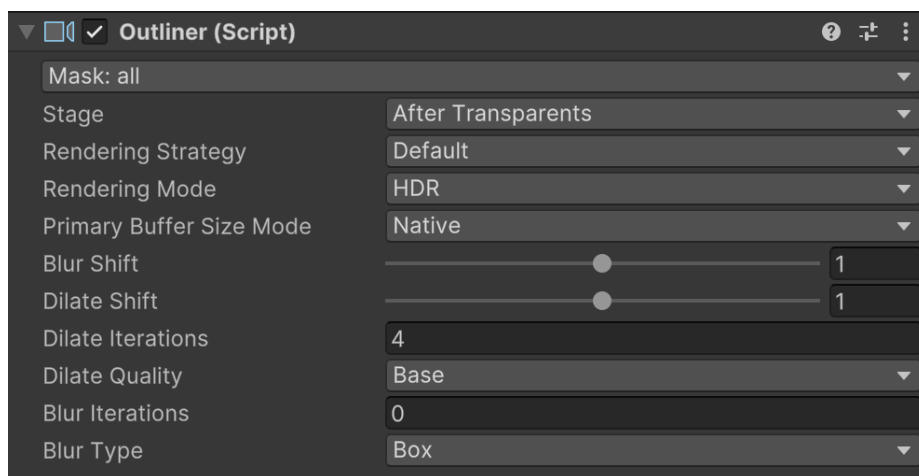
如果在安装过程中遇到问题，可以尝试以下方法：

- 重启 **Unity**: 导入资源后，重启 Unity 以确保所有内容正确加载。
- 重新导入: 如果发现某些内容缺失或无法正常工作，请尝试重新导入该资源包。
- 渲染器兼容性: 请确认在项目设置中选择了正确的渲染器 (Built-in、URP 或 HDRP)。
- 检查控制台: 安装完成后打开 Console 窗口，查看是否有错误或警告信息，这有助于定位潜在的设置问题。

如果问题仍然存在，欢迎通过电子邮件或 WeChat 支持与我们联系。

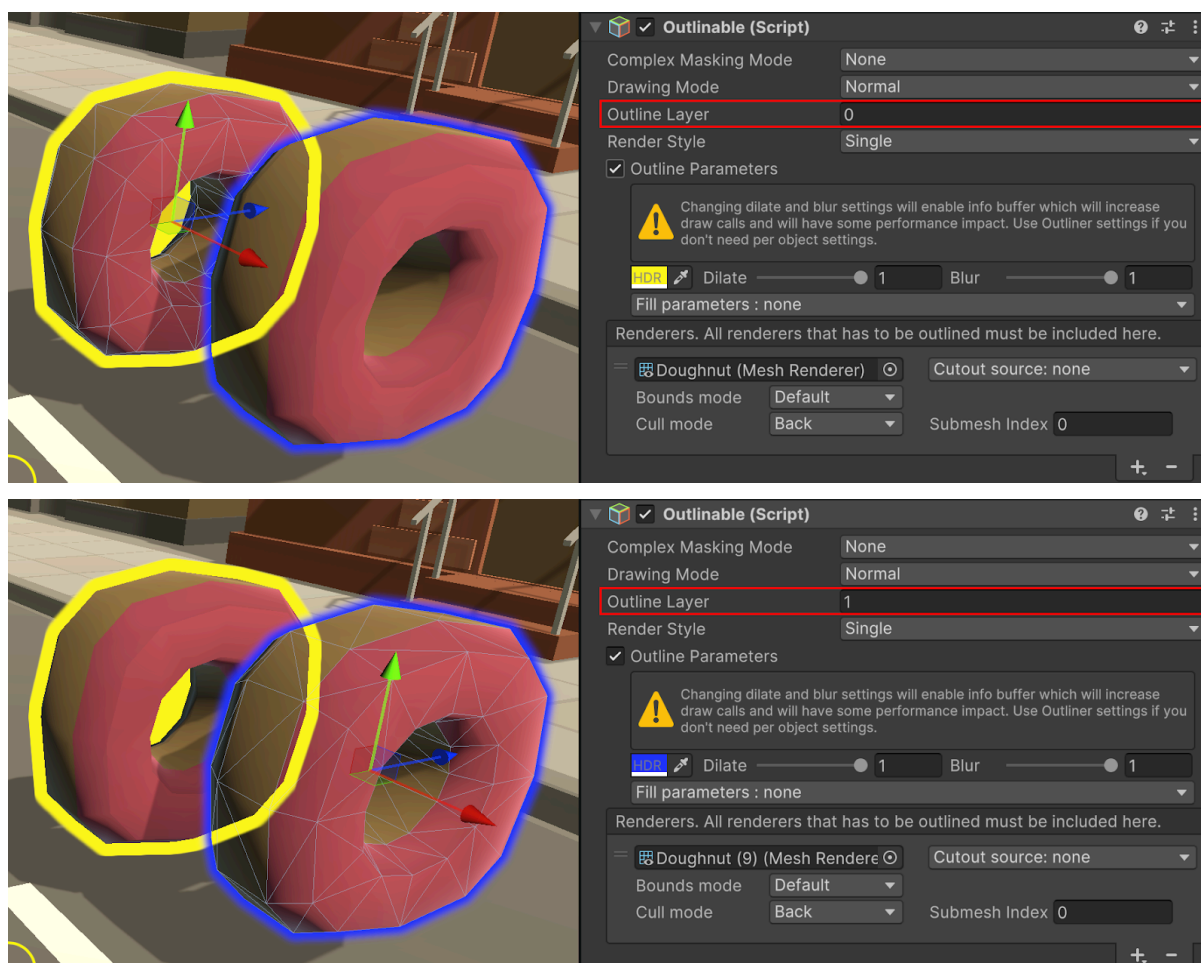
核心组件

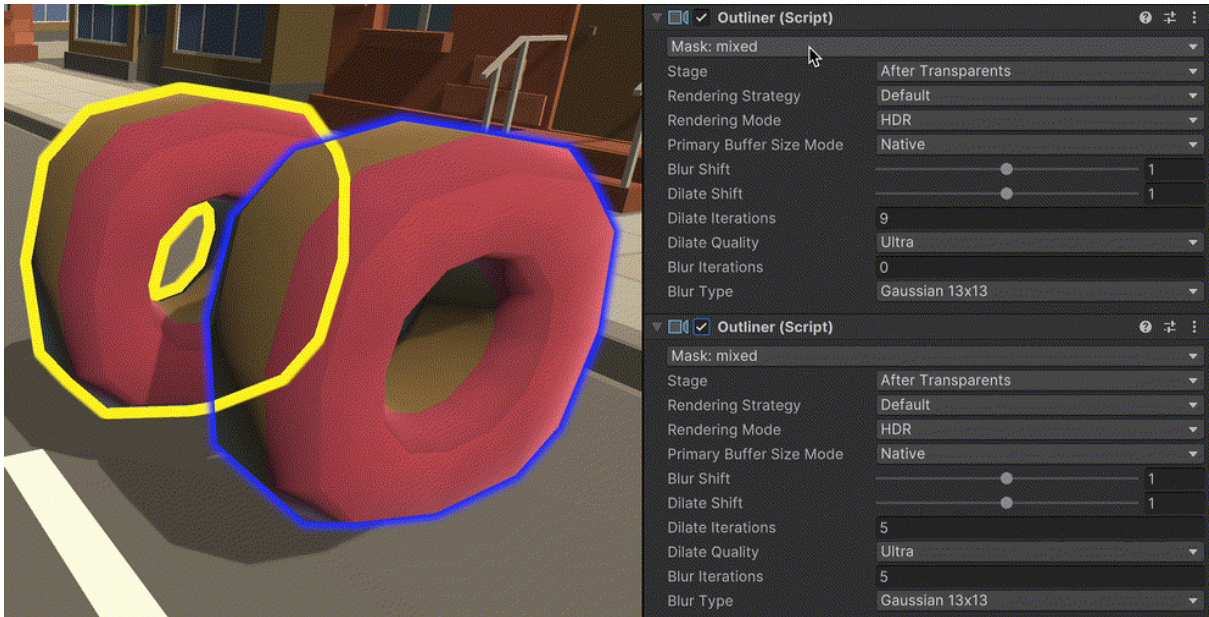
Outliner 设置



遮罩 (Mask)

用于定义 Outliner 应渲染哪些图层。如果你在 Outlinable 上选择了特定的图层，可以通过该 Outliner 将它们从渲染中排除。同一相机上可以添加多个 Outliner，并为它们设置不同的图层。





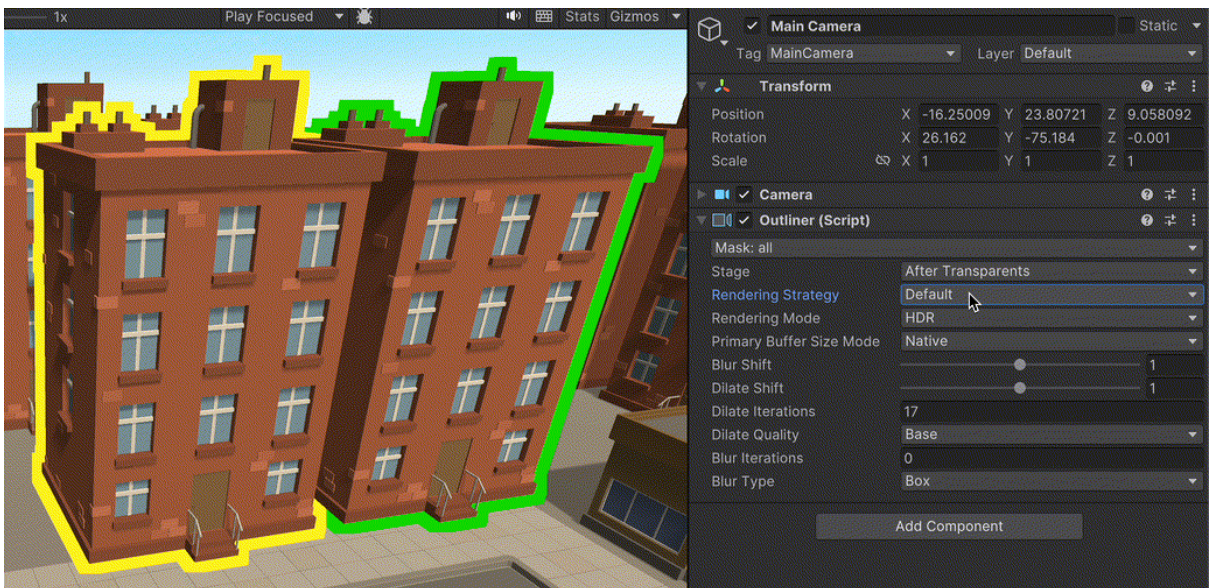
渲染阶段(Stage)

用于定义描边的渲染阶段。

- **Before Transparents:** 描边将在透明物体渲染之前绘制。
- **After Transparents:** 描边将在透明物体渲染之后绘制。

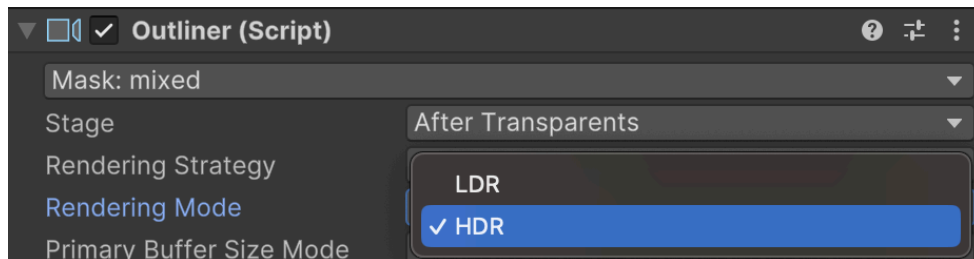
渲染策略(Rendering Strategy)

用于定义描边的渲染方式。在按对象渲染描边的情况下，各个对象的描边边缘不会相互融合。请注意，该方式会对性能产生一定影响。



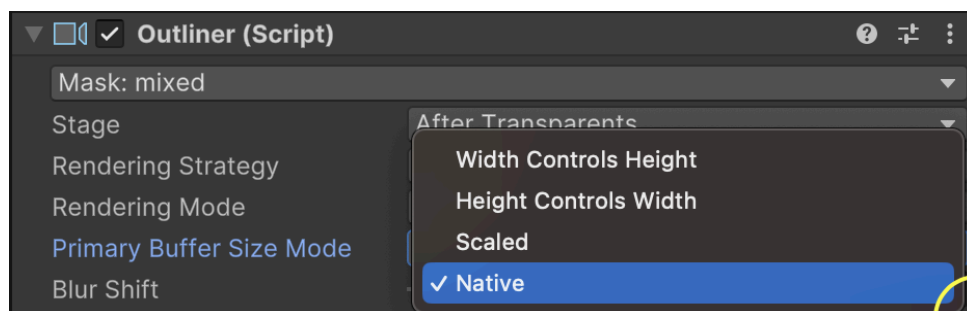
渲染模式 (Rendering Mode)

用于定义 Outliner 在渲染描边时使用 HDR 还是 LDR。在使用诸如 Bloom 等后处理效果时, HDR 非常重要;在其他情况下,使用 HDR 可能会造成资源浪费。请注意,如果相机不支持 HDR,或系统中禁用了 HDR,则不会使用 HDR。



主缓冲区尺寸模式 (Primary Buffer Size Mode)

该参数用于选择描边渲染分辨率的处理方式。

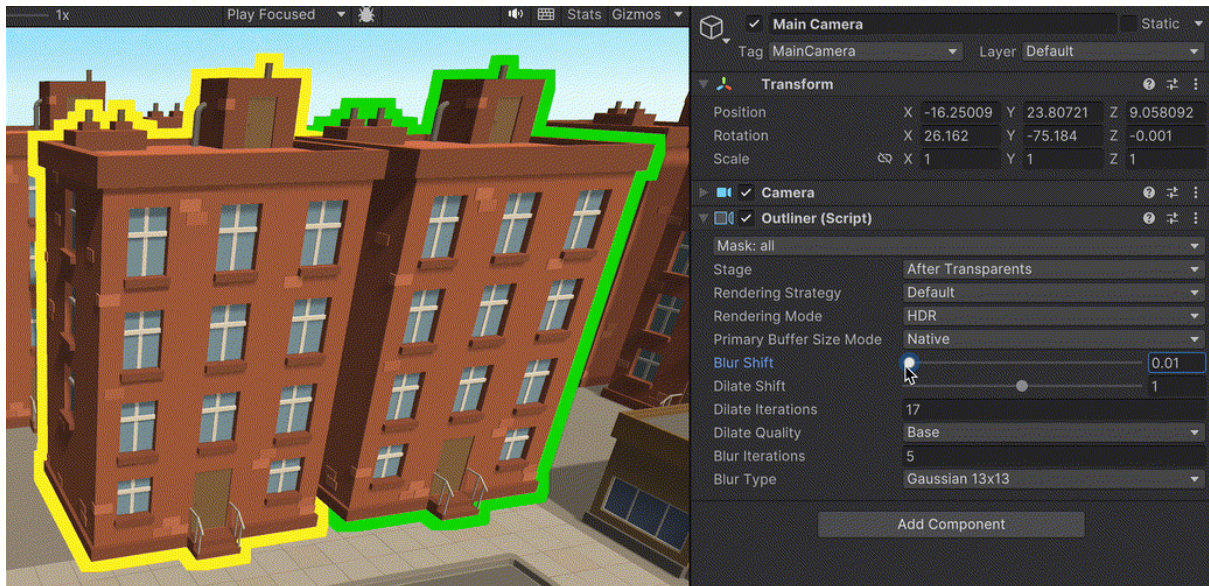


可用选项包括:

- **Native:** 描边以屏幕的原生分辨率进行渲染, 确保最高的画面质量。
- **Scaled:** 描边分辨率在水平和垂直方向上根据 **Primary Render Scale** 进行缩放。例如, 当屏幕分辨率为 800×600, 且 **Primary Render Scale** 设置为 0.5 时, 描边将以 400×300 的分辨率进行渲染。
建议在保证所需视觉质量的前提下, 将该值设置得尽可能低, 因为较低的数值可以提升性能。
- **Width Controls Height:** 描边的宽度基于 **Primary Size Reference** 进行设置, 高度将自动调整以保持正确的宽高比。
- **Height Controls Width:** 与上一种模式类似, 但此模式下高度基于 **Primary Size Reference** 进行设置, 宽度则按比例缩放。

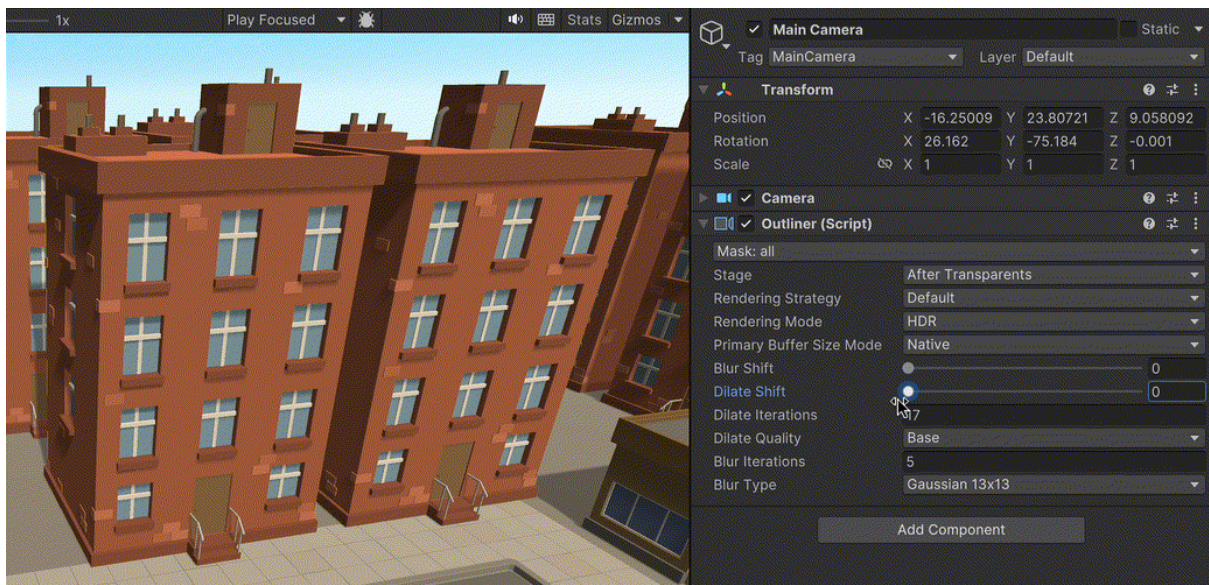
模糊偏移 (Blur Shift)

用于控制应用到最终图像上的模糊强度。请注意，数值大于 1 可能会产生伪影。



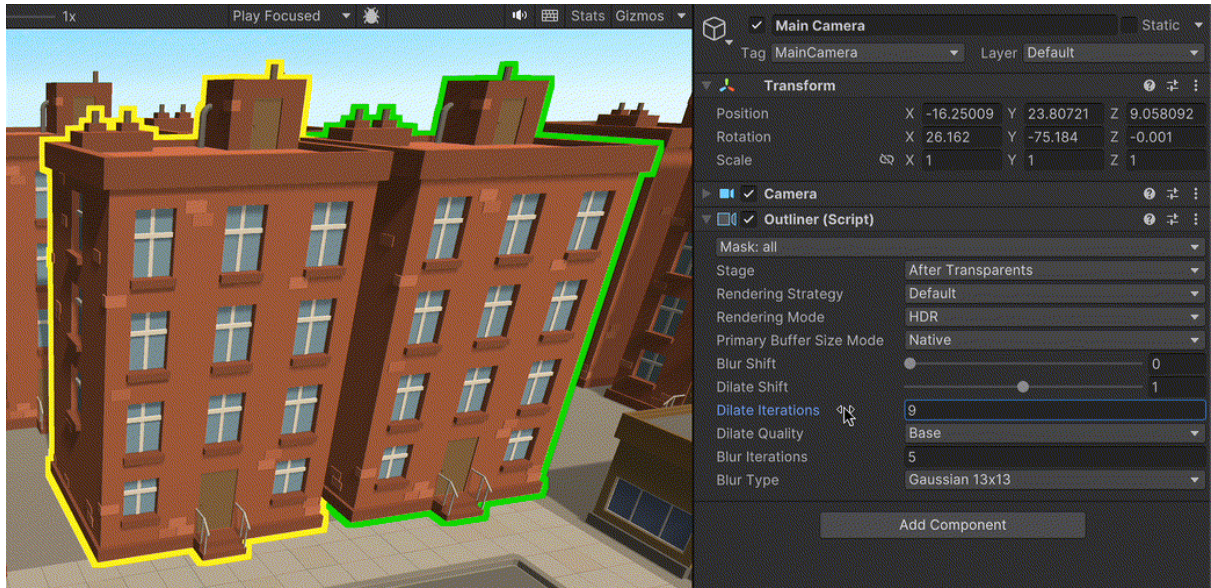
膨胀偏移 (Dilate Shift)

用于控制应用到最终图像上的膨胀程度。请注意，数值大于 1 可能会产生伪影。



膨胀迭代次数(Dilate Iterations)

用于定义膨胀效果需要应用多少次迭代。较小的数值可以提升性能,而较大的数值会产生更强的膨胀效果。**Shift** 与 **Iterations** 之间的区别与 **Blur Shift** 和 **Blur Iterations** 之间的区别相同。

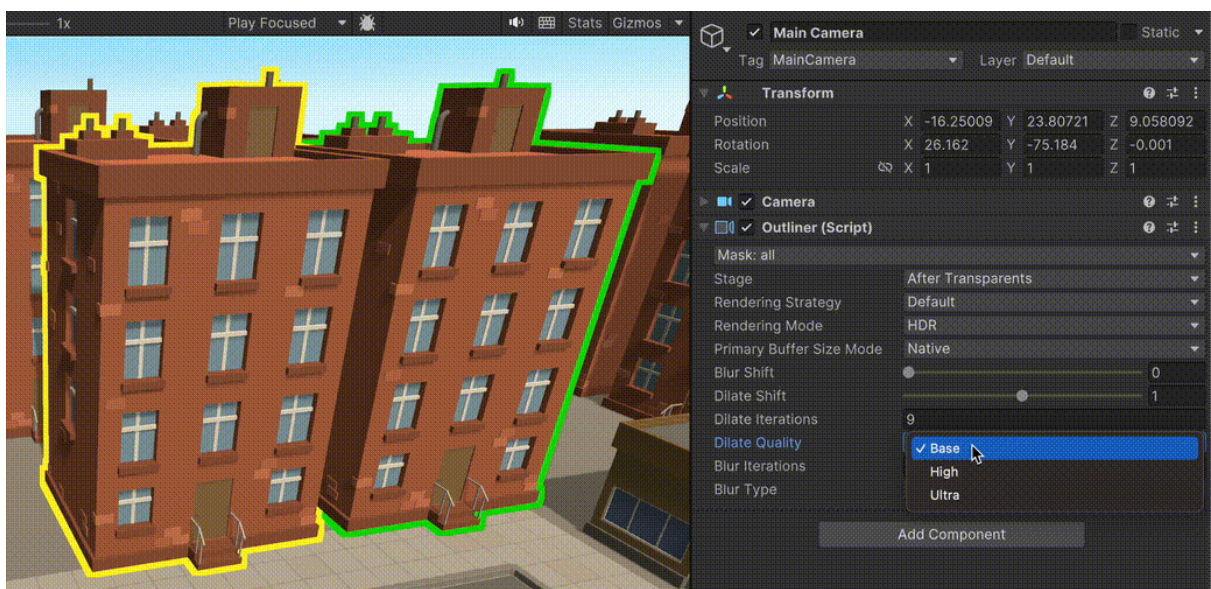


膨胀质量(Dilate Quality)

用于控制膨胀效果的质量。**Base** 设置具有最高的性能表现。**High** 和 **Ultra** 提供更高的质量, 但会消耗更多资源。

Base 变体会在水平和垂直两个方向上对描边进行膨胀, 效果相对近似, 但对性能更友好。另外两种变体采用圆形采样方式, 资源消耗更高, 但能够提供更精确的结果。

为了获得最佳性能, 建议在可能的情况下优先使用 **Base** 变体。



模糊迭代次数 (Blur Iterations)

用于定义模糊效果需要应用多少次迭代。较小的数值可以提升性能，而较大的数值会产生更强的模糊效果。

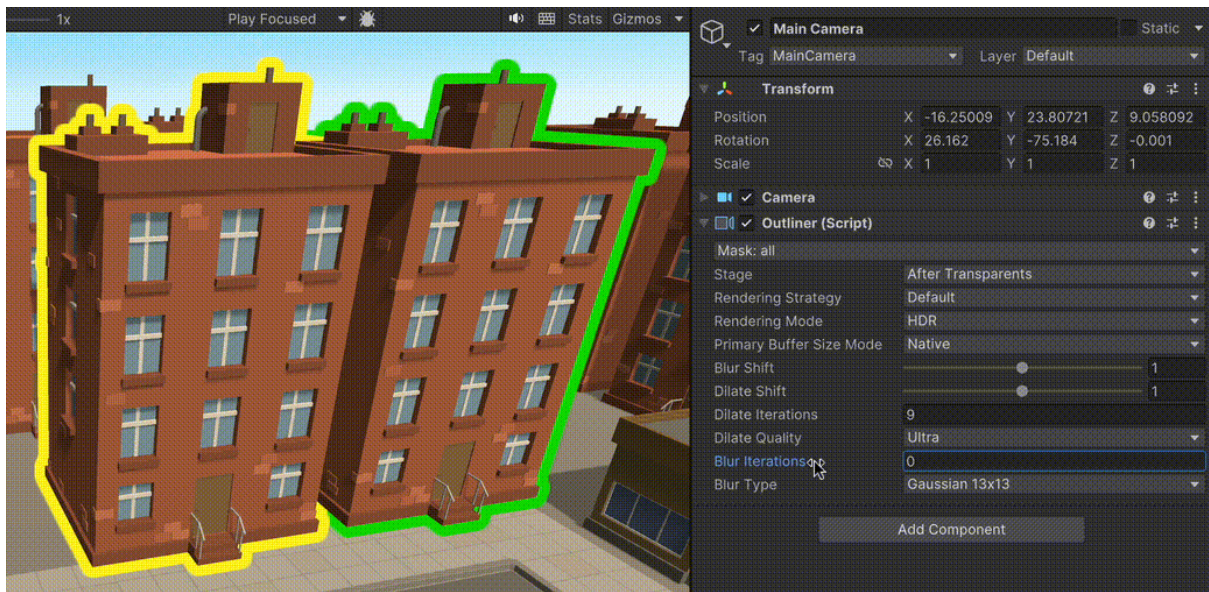
Shift 与 Iterations 之间的区别在于它们的作用方式：

Iterations 用于定义实现所需效果的后处理 Pass 次数，

而 Shift 则用于在其范围内调整效果强度。

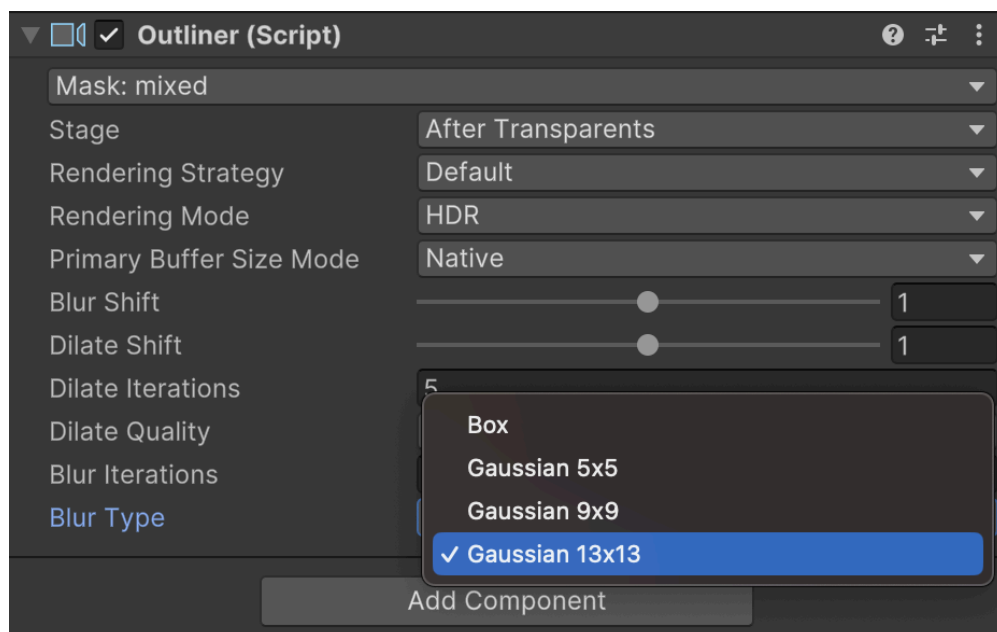
为了获得更好的性能，建议优先通过 Shift 参数来实现所需效果，

仅在确实必要时才增加 Iterations 的数值。

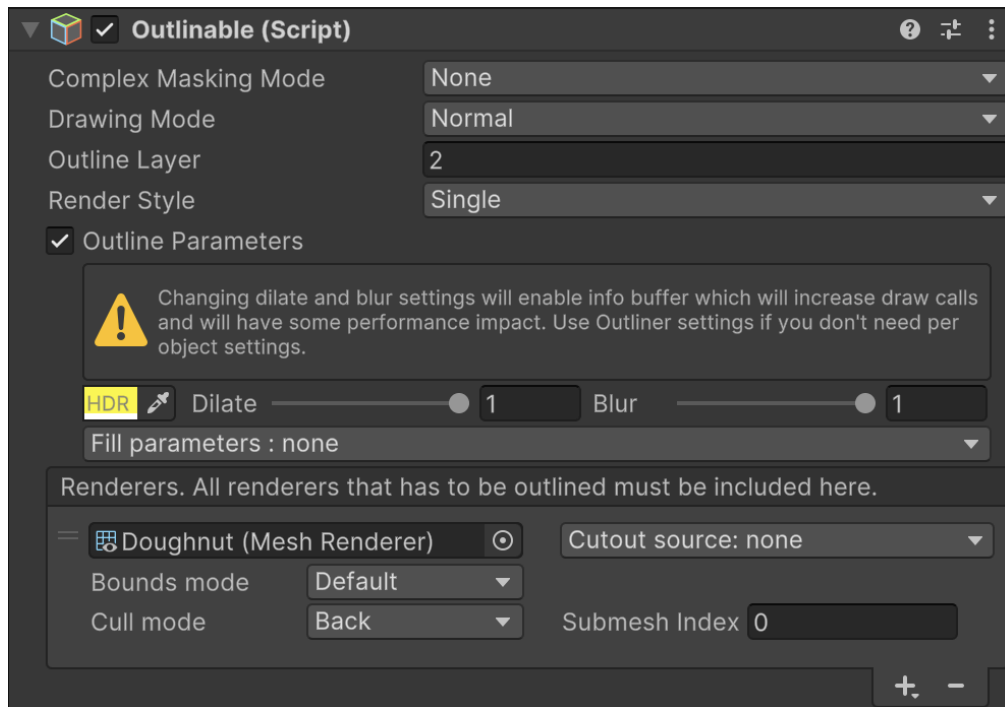


模糊类型 (Blur Type)

用于确定要应用的模糊类型。性能最高的选项是 **Box Blur**，而视觉效果最好的选项是 **Gaussian 13×13**。



Outlinable 设置



Outlinable 组件用于确定哪些对象会被描边系统处理。

复杂遮罩模式 (**Complex Masking Mode**) :

- **None**
不应用任何遮罩。
 - **Obstacle Mode**
描边仅在被标记为障碍物的对象后方可见。
 - **Masking Mode**
描边在被标记为遮罩的对象后方将被隐藏。
-

绘制模式 (**Drawing Mode**) :

- **Normal**
以默认方式绘制描边。
 - **Z Only**
仅向 Z 缓冲区写入。这对于通常不写入 Z 缓冲区的对象 (例如精灵或透明对象) 非常有用。
 - **Generic Mask**
使用遮罩绘制模型, 使描边在该区域完全不可见。
 - **Mask and Obstacle**
更多信息请参阅 遮罩 (Masking) 章节。
-

描边层级 (Outline Layer)

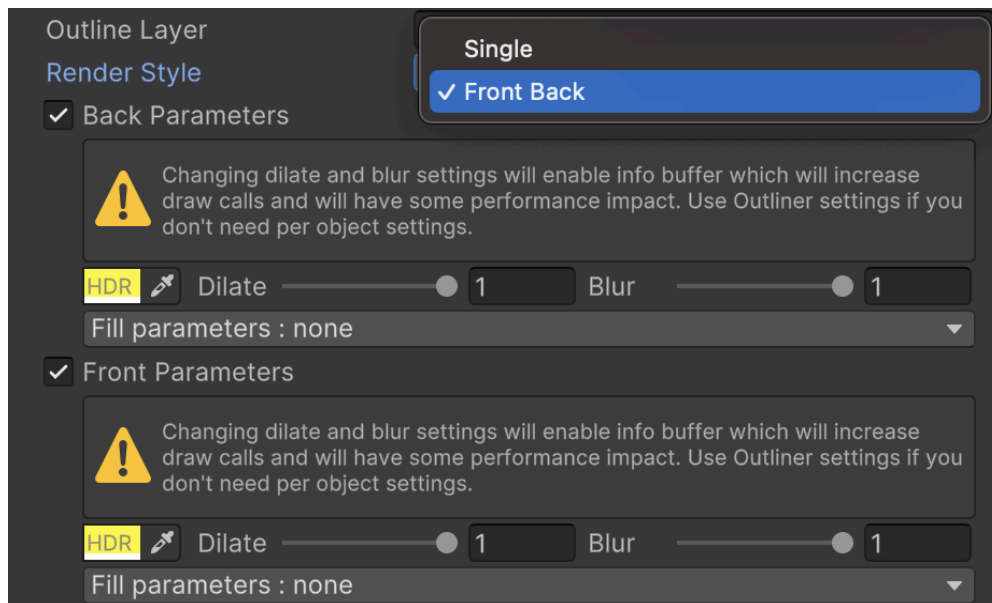
用于定义描边所属的层级。

如果你有可能相互重叠的独立描边效果，可以为 Outlinable 分配不同的层级，并添加多个 Outliner。

在这种情况下，各个描边效果之间不会相互干扰。

渲染样式 (Render Style)

用于确定是仅绘制通用描边，还是将描边 / 填充分离为被遮挡和未被遮挡两种状态。



后侧参数 (Back Parameters)

用于定义描边在被其他物体遮挡时 (例如在墙体后方) 所使用的参数。

前侧参数 (Front Parameters)

用于定义描边在直接可见 (未被任何物体遮挡) 时所使用的参数。

描边颜色 (Outline Color)

用于设置描边的颜色。

膨胀滑块 (Dilate Slider)

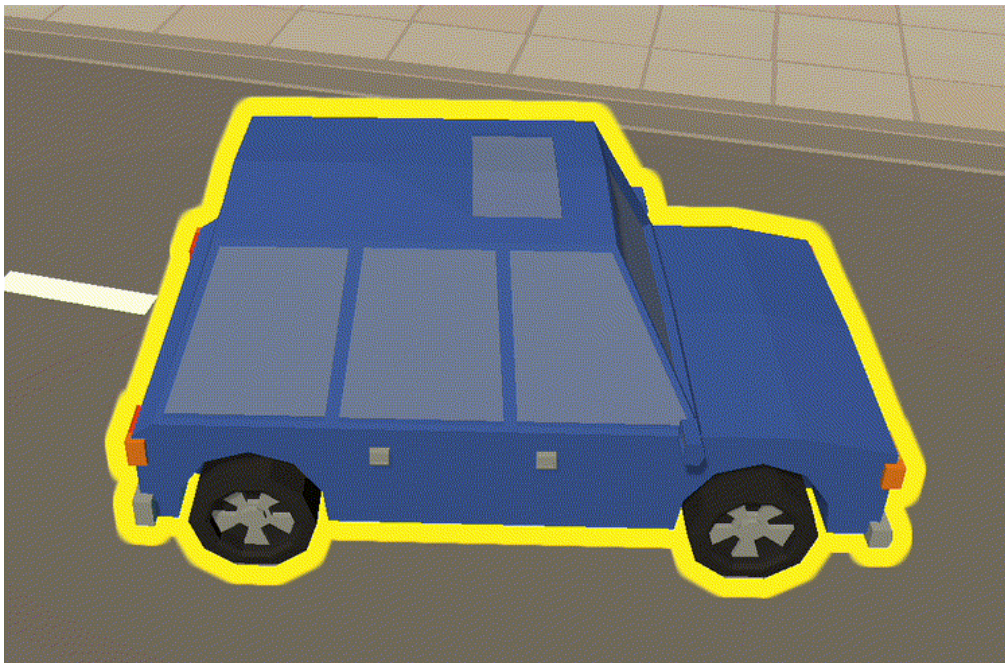
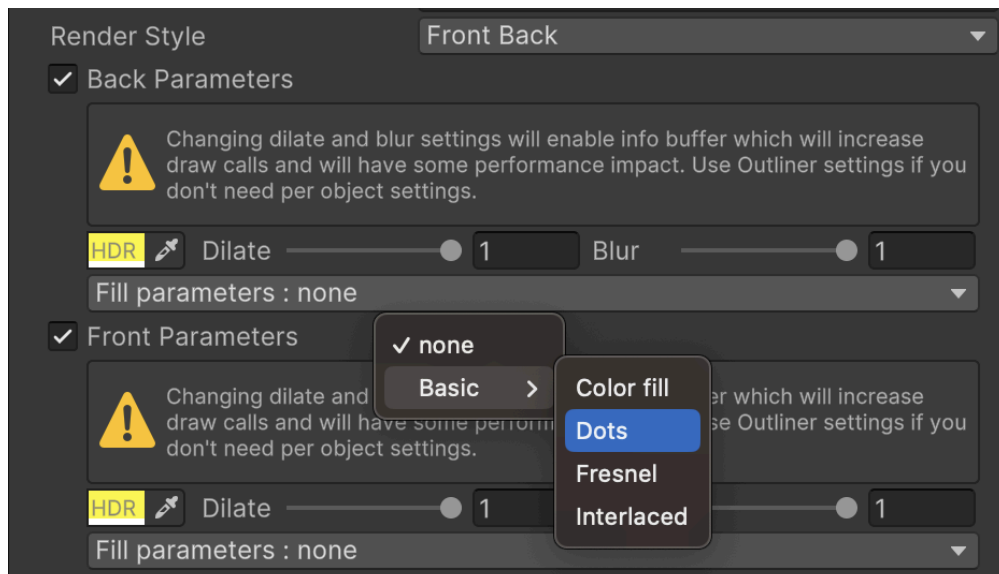
用于调整每个对象的膨胀偏移值，但这将启用信息缓冲区 (Info Buffer)。

模糊滑块 (Blur Slider)

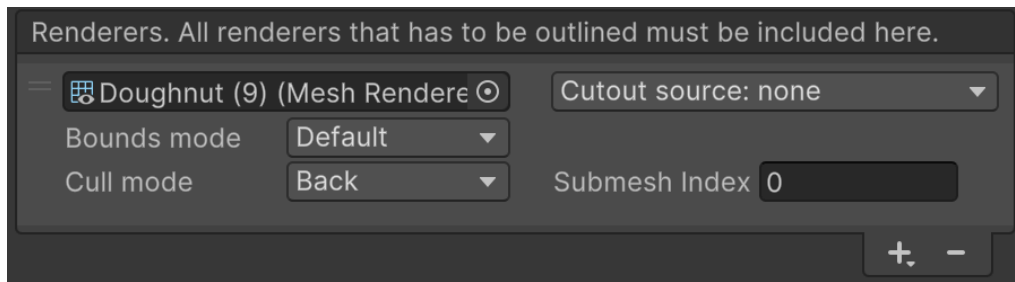
用于调整应用于每个对象的模糊强度，同样会启用信息缓冲区。

填充参数选择器(Fill Parameters Selector)

用于选择可用的填充类型。不同的填充类型可能具有不同的属性, 用于描述每个 Outlinable 的填充渲染方式。如果未选择任何类型, 则不会应用填充效果。

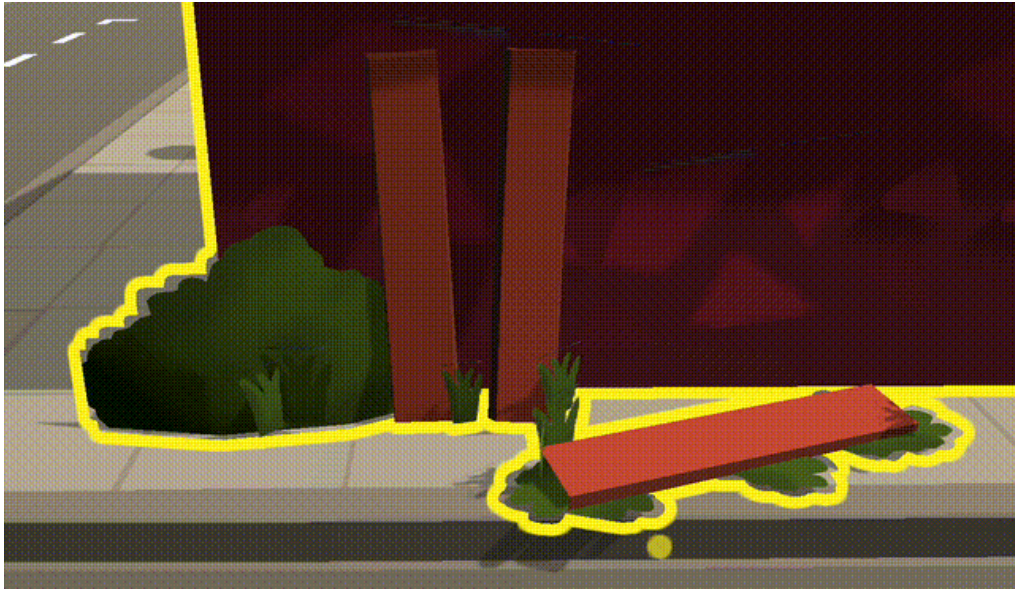


渲染器(Renderers)



- **渲染器(Renderer)**
用于生成描边的渲染器。通常应包含与你的模型 / 对象相关的所有渲染器。
被禁用的渲染器将不会被渲染。
- **包围盒模式(Bounds Mode)**
用于定义渲染器包围盒的计算方式：
 - **Default:** 直接使用渲染器提供的包围盒。
 - **Force Recalculate:** 每一帧重新计算包围盒, 适用于动态更新的对象。
 - **Manual:** 允许用户手动定义自定义包围盒。
- **剔除模式(Cull Mode)**
用于定义渲染 Outlinable 目标时使用的剔除方式。
通常 **Back** 是最合适的选择。如果几何体需要从正反两面都可见,
请选择 **Off**。**Front** 适用于极端情况, 通常很少需要, 但在必要时也可以使用。
- **裁剪来源(Cutout Source)**
用于指定作为裁剪 Alpha 来源的纹理。选择纹理后, 将会显示额外的控制选项
 - **Cutout Threshold:** 用于设置纹理被渲染所需的最小 Alpha 值。Alpha 值低于该阈值的像素将被丢弃。
 - **Texture Index:** 适用于纹理数组, 可指定数组中使用的具体纹理索引。
 - **Cutout Mask:** 用于指定纹理的哪个通道(例如 R、G、B 或 A)
将用于裁剪比较。

这些选项为自定义裁剪行为提供了灵活性, 以满足不同的使用需求。



- 子网格索引(**Submesh Index**)
用于指定用于渲染描边部分的子网格索引。

高级设置

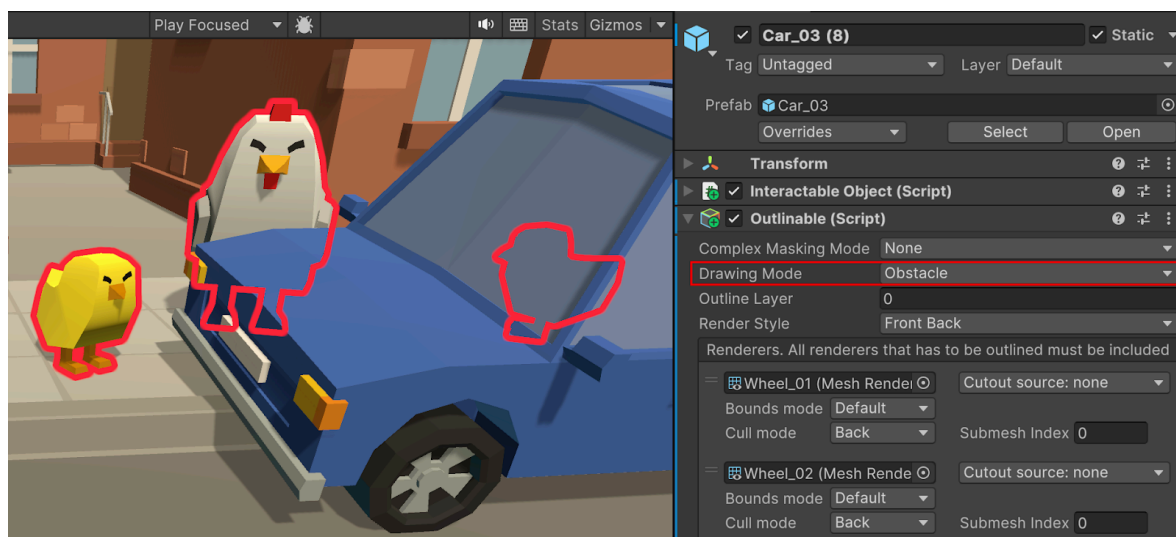
遮罩

Obstacle Mode

在此模式下，描边仅在被标记为障碍物的对象后方可见。

这有助于强调被描边对象位于其他物体之后，从而营造出遮挡与层次关系。

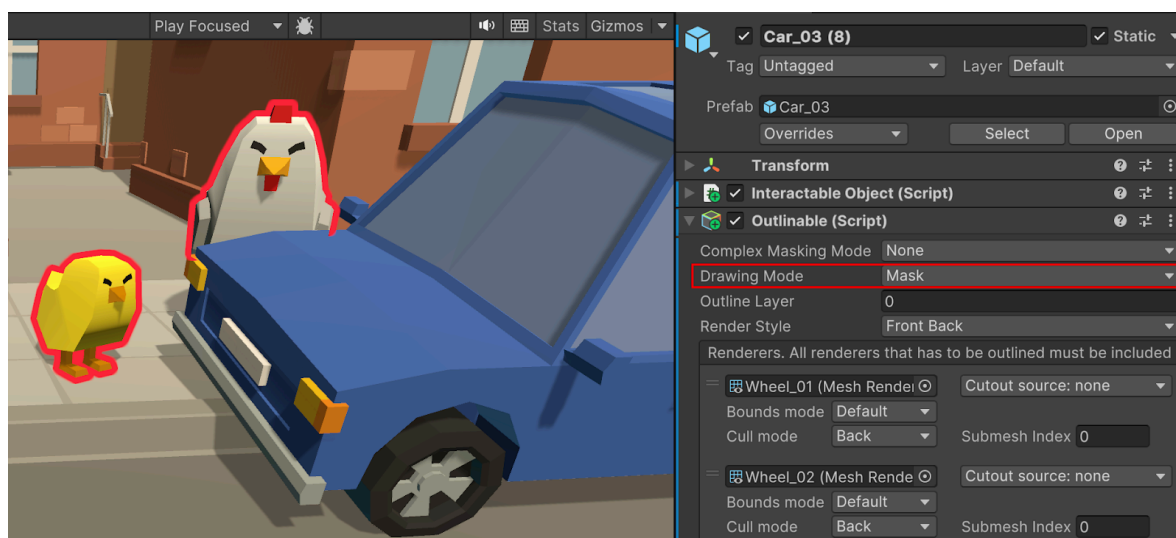
例如，角色的描边仅会在汽车后方显示。



Masking Mode

在此模式下，描边会在被标记为遮罩的对象后方被隐藏。

该模式用于确保描边不会显示在场景中的某些元素后面，从而获得更加干净、精确的视觉效果。例如，角色的描边会在汽车后方被隐藏。



每种模式都可以根据场景的需求实现不同的视觉效果。

2D 功能

2D 精灵及类似元素通常不会向 Z 缓冲区写入数据，而 Z 缓冲区对于前后渲染模式的正常工作至关重要。

为了解决这一问题，可以将某些发生重叠的对象以 **Z Only** 模式进行渲染，强制它们写入 Z 缓冲区。

- 在未写入 Z 缓冲区的情况下：
由于未参与深度计算，对象在前后渲染模式下可能无法正确显示。



- 在写入 Z 缓冲区的情况下：
对象会被正确地纳入深度缓冲区计算，
从而在前后渲染模式下实现正确的渲染和层级关系。



集成

使用 C# API

你可以通过脚本轻松地与 Outlinable 组件进行交互：

1. 要访问挂载在同一 GameObject 上的组件：

```
using EPOOutline;
using UnityEngine;

public class MyScript : MonoBehaviour
{
    private void Start()
    {
        var outlinableToUse = GetComponent<Outlinable>();
    }
}
```

2. 在运行时添加 Outlinable 组件：

```
using EPOOutline;
using UnityEngine;

public class MyScript : MonoBehaviour
{
    private void Start()
    {
        var outlinableToUse = gameObject.AddComponent<Outlinable>();
    }
}
```

3. Outlinable 组件的关键属性：

- 设置渲染样式：

```
// Render both behind and in front
outlinable.RenderStyle = RenderStyle.FrontBack;

// Render regardless of being obscured
outlinable.RenderStyle = RenderStyle.Single;
```

- 绘制模式：

```
// Normal rendering
outlinable.DrawingMode = OutlinableDrawingMode.Normal;

// Z-buffer only
outlinable.DrawingMode = OutlinableDrawingMode.ZOnly;

// Both
outlinable.DrawingMode = OutlinableDrawingMode.Normal |
OutlinableDrawingMode.ZOnly;
```

- 配置描边参数：

```
// Outline color
outlinable.OutlineParameters.Color = Color.green;

// Blur shift
outlinable.OutlineParameters.BlurShift = 0.5f;

// Dilate shift
outlinable.OutlineParameters.DilateShift = 0.5f;

// Disable outline
outlinable.OutlineParameters.Enabled = false;
```

- 管理层级与渲染目标：

```
// Enable layer 3
outlinable.OutlineLayer = 3;

// Add a renderer
outlinable.OutlineTargets.Add(new OutlineTarget(GetComponent<Renderer>()));
```

- 填充模式设置：

```
outlinable.OutlineParameters.FillPass.Shader = Resources.Load<Shader>("Easy
performant outline/Shaders/Fills/ColorFill");

// Fill color
outlinable.OutlineParameters.FillPass.SetColor("_PublicColor", Color.yellow);
```

还提供了更多示例，用于配置前侧和后侧描边参数。

DOTween 支持

要启用 DOTween 支持, 请按照以下步骤操作:

1. 打开 DOTween 设置。
2. 在 DOTween 设置中直接启用描边支持。

要在脚本中使用 DOTween, 请包含以下内容:

```
using DG.Tweening;
```

随后, 你可以轻松地对大多数 Outlinable 参数进行动画处理。例如:

```
// Animate color
outlinable.FrontParameters.DOColor(new Color(0, 0, 1, 0), 0.5f);

// Animate dilate shift
outlinable.FrontParameters.DODilateShift(1.0f, 0.5f);

// Animate blur shift
outlinable.FrontParameters.DOBlurShift(0.0f, 0.5f);
```

常见问题(FAQ)

问. **EP Outline** 是否适合初学者使用？

答. 是的, 没有任何复杂之处。完成初始设置后, 描边系统只需要使用 2 个组件即可。

问. 支持哪些平台？

答. Android、iOS、PC、Mac、PS4、Oculus/Vive、WebGL。
其他平台理论上也可以使用, 但我们尚未进行测试。

问. 演示场景不完整, 我在视频中看到的版本在哪里可以获取？

答. 由于与部分免费资源开发者存在分歧, 我们无法加入完整版本。
请通过电子邮件 pirate.parrot.software@gmail.com 或 WeChat ID: **thepirateparrot** 联系我以获取帮助, 同时请务必提供你的发票编号。

问. 我可以在商业产品中使用你们的演示场景吗？

答. 可以。演示场景中包含的所有模型均来自 Asset Store 的免费资源。

问. **EP Outline** 是否支持 HDRP？

答. 是的, 支持 HDRP。

问. 导入后出现了一些错误, 为什么？

答. 首先, 尝试重启 Unity 并再次检查。其次, 尝试重新导入该资源。
如果仍无法解决, 请通过电子邮件 pirate.parrot.software@gmail.com 或 WeChat ID: **thepirateparrot** 与我联系。

问. **Outliner** 组件的作用是什么？

答. 它作为通用设置的提供者。你可以在[此处](#)找到更多信息。

问. 添加描边后 **FPS** 降低了, 我该怎么办?

答. 请先确认性能问题是否由描边本身引起。你可以使用 Profiler 来定位导致卡顿的原因。尝试禁用 Outliner, 观察帧率是否有明显变化。可以尝试降低主缓冲区分辨率, 或减少膨胀 / 模糊的迭代次数。同时请务必在目标设备上进行测试, 因为 Unity 编辑器本身会对性能产生较大影响。如果以上方法都无法解决问题, 请通过电子邮件 pirate.parrot.software@gmail.com 或 WeChat ID: thepirateparrot 与我联系。

问. 导入描边后模型变成了粉色, 这是怎么回事?

答. 这种情况通常发生在为未正确配置的项目添加了 URP / HDRP 支持时。解决方法是: 从 Player Settings 中的 Quality 和 Graphics 设置里移除 URP / HDRP Renderer, 或通过 **Edit > Render Pipeline > Universal Render Pipeline > Upgrade Project Materials to UniversalRP Materials** 升级材质。如果问题仍然存在, 请通过电子邮件 pirate.parrot.software@gmail.com 或 WeChat ID: thepirateparrot 联系我。

问. 如何在不破坏项目的情况下删除描边插件?

答. 只需将其从 plugins 文件夹中移除即可, 项目不会受到影响。

问. 如何修改描边的宽度?

答. 你可以在 Outliner 中调整膨胀迭代次数 (Dilate Iterations) 和膨胀偏移 (Dilate Shift) 参数。更多信息请参见 [此处](#)。

联系信息

1. [Asset Store Page](#)
2. Email: pirate.parrot.software@gmail.com
3. WeChat ID: thepirateparrot

如有任何问题、功能建议或反馈, 欢迎通过电子邮件或 WeChat 与我们联系！

★ 通过评价支持我们！ ★

如果你觉得这份文档对你有帮助, 欢迎抽出一点时间留下评价！

你的反馈对我们改进 [Easy Performant Outline](#) 并添加新功能非常重要。

每一条评价都意义非凡——感谢你的支持！